LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 9 – STATIC & COLLECTION



NANDA MAYLA ADIASTI

103132400003

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Pengenalan Static

   Dalam Pemrograman Berorientasi Objek, kata kunci static digunakan untuk membuat variabel atau method yang melekat langsung pada suatu kelas, bukan pada objek instansiasi tertentu. Artinya, anggota yang bersifat static dapat diakses secara langsung melalui kelasnya tanpa harus membuat objek baru terlebih dahulu.

   A. Static Variabel

      Static variable (atau variabel kelas) adalah variabel yang dimiliki oleh suatu kelas dan dibagi bersama oleh semua objek instansiasi dari kelas tersebut. Jika nilai dari variabel statis ini diubah oleh suatu objek, maka perubahan tersebut akan terlihat oleh semua objek lainnya. Variabel lokal di dalam method tidak dapat didefinisikan sebagai static.

   B. Static Method

      Static method adalah method yang didefinisikan sebagai bagian dari kelas itu sendiri. Karakteristik utamanya meliputi:

      a. Tidak memerlukan instansiasi objek untuk dapat dipanggil atau dijalankan.

      b. Tidak dapat menjalankan atau memanggil method yang bukan static.

      c. Tidak dapat mengakses atau mengacu pada variabel yang bukan static.

2. Collection Framework

   Collection adalah struktur data berupa objek khusus di dalam Java yang digunakan untuk menampung sekumpulan objek dengan berbagai tipe data. Karena semua kelas di Java merupakan turunan dari kelas Object, maka dengan menerapkan prinsip polimorfisme, collection mampu menampung objek dengan tipe apapun.

   Collection didefinisikan sebagai sebuah interface di mana terdapat berbagai kelas turunan yang mengimplementasikannya. Hierarki utama dari collection mencakup beberapa interface turunan seperti List, Queue, dan Set.

   A. Generics

      Dengan menggunakan generics (ditandai dengan simbol sudut < >), kita dapat membatasi atau menentukan secara spesifik tipe data apa saja yang dapat ditampung oleh suatu collection sehingga program menjadi lebih aman dari kesalahan tipe data saat waktu kompilasi.

   B. Sorting pada Collection

      Pengurutan elemen di dalam collection dapat dilakukan melalui dua cara utama:

      a. Interface Comparable: Digunakan untuk pengurutan berdasarkan satu properti utama. Kelas yang elemennya ingin diurutkan harus mengimplementasikan interface Comparable dan meng-override method compareTo(Object obj). Method ini mengembalikan nilai negatif, nol, atau positif tergantung pada perbandingan antar objek.

b. Interface Comparator: Digunakan apabila kita ingin melakukan pengurutan berdasarkan banyak properti (multiple property) atau kriteria kustom. Caranya adalah dengan membuat kelas terpisah yang mengimplementasikan interface Comparator<T> serta meng-override method compare(T obj1, T obj2).

C. Filtering pada Collection

a. Cara Konvensional: Menggunakan struktur kontrol perulangan (seperti for-each) yang dikombinasikan dengan percabangan if untuk memeriksa kriteria tertentu pada setiap elemen.

b. Ekspresi Lambda dan Stream API: Cara modern yang lebih ringkas dan deklaratif dengan memanfaatkan method .stream().filter(...) untuk menyaring data sesuai kondisi yang diinginkan.

B. Guided

1. Guided 1

[Share.java]

```
package Guided.Guided1;

public class Share {
    private int privateInt;
    private static int staticInt;

    public Share(int pr, int si){
        privateInt = pr;
        staticInt = si;
    }

    public String toString(){
        return privateInt + " " + staticInt;
    }
}
```

[Drive.java]

```
package Guided.Guided1;


public class Driver {
    public static void main(String[] args){
        Share s1 = new Share(4, 4);
        System.out.println(s1.toString());

        Share s2 = new Share(8, 2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());

        Share s3 = new Share(6, 22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
4 4
4 2
8 2
4 22
8 22
6 22
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.231 s
Finished at: 2026-11-10T12:57:20+07:00
------------------------------------------------------------------------
```

2. Guided 2

[CounterMachine.java]

```
package Guided.Guided2;

public class CounterMachine {
    static int counter = 0;

    public static void count(){
        counter++;
    }
}
```

[Driver.java]

```
package Guided.Guided2;

public class Driver {
    public static void main(String[] args) {
        for(int i = 0; i<10; i++){
            if(i%2==0){
                CounterMachine.count();
            }
        }

        System.out.println(CounterMachine.counter);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
5
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.024 s
Finished at: 2026-11-10T13:15:04+07:00
------------------------------------------------------------------------
```

3. **Guided 3**

[Iteration.java]

```
package Guided.Guided3;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Iteration {
    public static void main(String[] args) {
        // 1. Inisialisasi List bertipe String
        List<String> listBuah = new ArrayList<>();

        // 2. Menambah elemen ke dalam List
        listBuah.add("Apel");
        listBuah.add("Mangga");
        listBuah.add("Jeruk");
        listBuah.add("Pisang");

        // 3. Operasi utilitas List
        System.out.println("Total elemen: " + listBuah.size());
        System.out.println("Apakah ada 'Mangga'? " +
listBuah.contains("Mangga"));

        // Menghapus data
        listBuah.remove("Pisang");

        // --- TEKNIK 1: For-Loop Tradisional (Indeks) ---
        System.out.println("\n--- 1. Normal For-Loop (Indeks) ---");
        for (int i = 0; i < listBuah.size(); i++) {
            System.out.println("Index " + i + ": " + listBuah.get(i));
        }

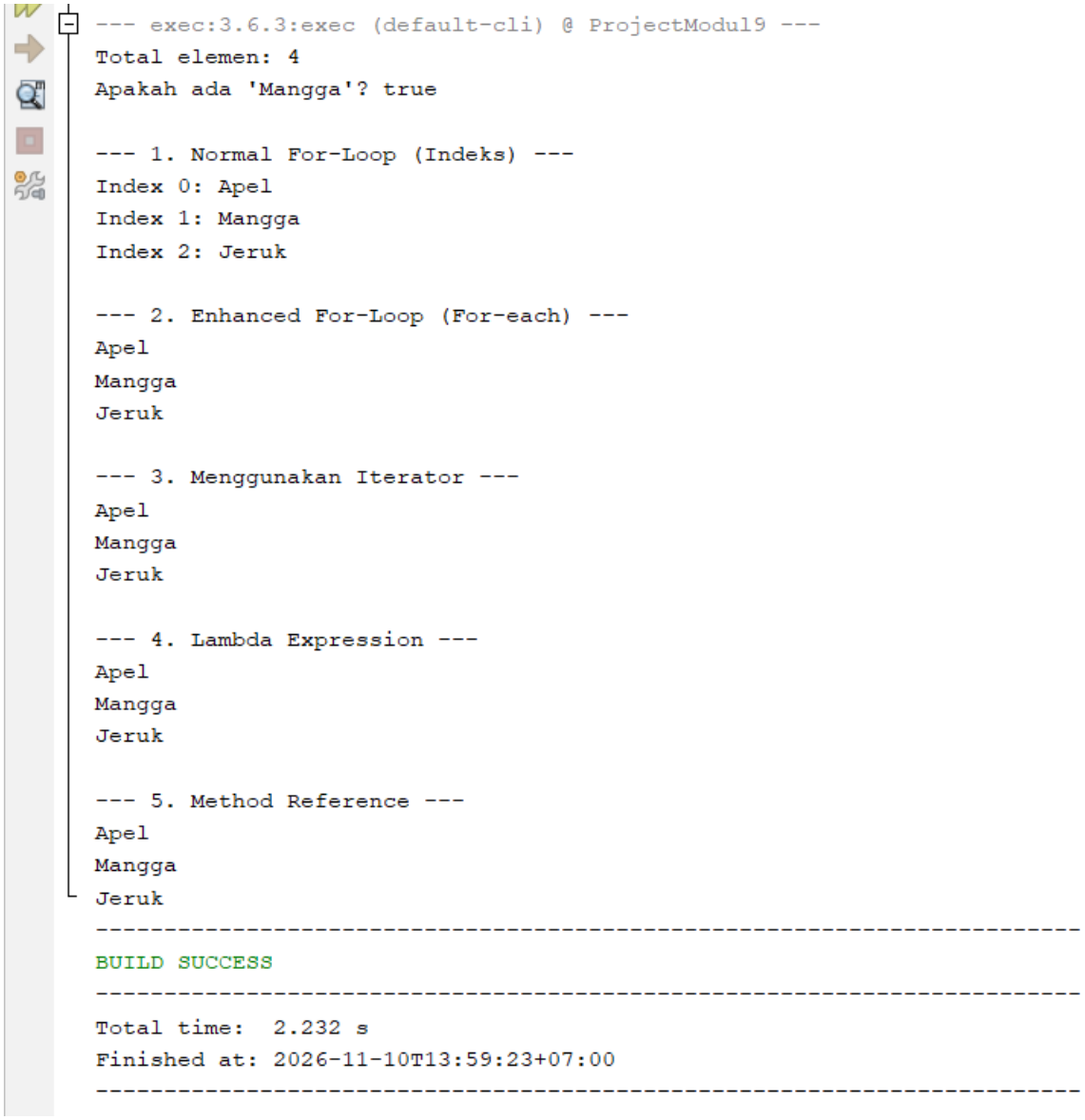
        // --- TEKNIK 2: Enhanced For-Loop (For-Each) ---
        System.out.println("\n--- 2. Enhanced For-Loop (For-each) ---");
        for (String buah : listBuah) {
            System.out.println(buah);
        }

        // --- TEKNIK 3: Iterator Interface ---
        System.out.println("\n--- 3. Menggunakan Iterator ---");
        Iterator<String> itr = listBuah.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
```

```
            // --- TEKNIK 4: Lambda Expression (Java 8+) ---
            System.out.println("\n--- 4. Lambda Expression ---");
            listBuah.forEach(item -> System.out.println(item));

            // --- TEKNIK 5: Method Reference (Java 8+) ---
            System.out.println("\n--- 5. Method Reference ---");
            listBuah.forEach(System.out::println);
        }
}
```

Output:



```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
Total elemen: 4
Apakah ada 'Mangga'? true

--- 1. Normal For-Loop (Indeks) ---
Index 0: Apel
Index 1: Mangga
Index 2: Jeruk

--- 2. Enhanced For-Loop (For-each) ---
Apel
Mangga
Jeruk

--- 3. Menggunakan Iterator ---
Apel
Mangga
Jeruk

--- 4. Lambda Expression ---
Apel
Mangga
Jeruk

--- 5. Method Reference ---
Apel
Mangga
Jeruk
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.232 s
Finished at: 2026-11-10T13:59:23+07:00
------------------------------------------------------------------------
```

4. **Guided 4**

[Employee.java]

```
package Guided.Guided4.Sorting;

public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;

    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public int getSalary() {
        return salary;
    }

    // Default sorting: Berdasarkan nama (A-Z)
    @Override
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return String.format("Employee[name=%-6s, salary=%d]", name,
salary);
    }
}
```

[SalaryComparator.java]

```
package Guided.Guided4.Sorting;

import java.util.Comparator;

public class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return Integer.compare(e1.getSalary(), e2.getSalary());
    }
}
```

[Main.java]

```
package Guided.Guided4.Sorting;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList<>();
        listEmp.add(new Employee("Bobby", 3000));
        listEmp.add(new Employee("Erick", 1600));
        listEmp.add(new Employee("Rey", 2500));
        listEmp.add(new Employee("Anna", 3500));

        System.out.println("--- Data Awal ---");
        listEmp.forEach(System.out::println);

        // Sorting 1: Natural Order via Comparable (compareTo)
        Collections.sort(listEmp);
        System.out.println("\n--- Sorted by Name (Comparable / Natural
Order) ---");
        listEmp.forEach(System.out::println);

        // Sorting 2: Custom Order via Comparator (compare)
        Collections.sort(listEmp, new SalaryComparator());
        System.out.println("\n--- Sorted by Salary (Comparator / Custom
Order) ---");
        listEmp.forEach(System.out::println);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
--- Data Awal ---
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Anna  , salary=3500]

--- Sorted by Name (Comparable / Natural Order) ---
Employee[name=Anna  , salary=3500]
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]

--- Sorted by Salary (Comparator / Custom Order) ---
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Bobby , salary=3000]
Employee[name=Anna  , salary=3500]
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.141 s
Finished at: 2026-11-10T14:02:44+07:00
------------------------------------------------------------------------
```

5. Guided 5

[Mahasiswa.java]

```
package Guided.Guided5.Filtering;

public class Mahasiswa {
    private String nim;
    private String nama;
    private double ipk;

    public Mahasiswa(String nim, String nama, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.ipk = ipk;
    }

    public String getNim() {
        return nim;
    }

    public String getNama() {
        return nama;
    }

    public double getIpk() {
        return ipk;
    }

    @Override
    public String toString() {
        return String.format("Mhs[NIM=%s, Nama=%-6s, IPK=%.2f]", nim,
nama, ipk);
    }
}
```

[MainFilter.java]

```
package Guided.Guided5.Filtering;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class MainFilter {
    public static void main(String[] args) {
        List<Mahasiswa> list = new ArrayList<>();
        list.add(new Mahasiswa("001", "Bobby", 3.25));
        list.add(new Mahasiswa("002", "Erick", 2.80));
```

```
        list.add(new Mahasiswa("003", "Rey", 3.75));
        list.add(new Mahasiswa("004", "Anna", 3.90));

        System.out.println("=== 1. FILTERING (IPK >= 3.50) ===");

        // --- Pendekatan 1: Imperatif (Konvensional) ---
        System.out.println("Cara Konvensional:");
        List<Mahasiswa> mhsCumlaudeKonven = new ArrayList<>();
        for (Mahasiswa m : list) {
            if (m.getIpk() >= 3.50) {
                mhsCumlaudeKonven.add(m);
            }
        }
        for (Mahasiswa m : mhsCumlaudeKonven) {
            System.out.println(m);
        }

        // --- Pendekatan 2: Deklaratif (Stream API) ---
        System.out.println("\nCara Stream API:");
        List<Mahasiswa> mhsCumlaudeStream = list.stream()
                .filter(m -> m.getIpk() >= 3.50)
                .collect(Collectors.toList());
        mhsCumlaudeStream.forEach(System.out::println);

        System.out.println("\n=== 2. SEARCHING / PENCARIAN
    TUNGGAL ===");

        // --- Pencarian Konvensional ---
        System.out.println("Cara Konvensional (Cari 'Erick'):");
        Mahasiswa targetKonven = null;
        for (Mahasiswa m : list) {
            if (m.getNama().equalsIgnoreCase("Erick")) {
                targetKonven = m;
                break; // Hentikan loop saat ditemukan
            }
        }
        System.out.println(targetKonven != null ? targetKonven : "Data tidak
    ditemukan");

        // --- Pencarian via Stream API ---
        System.out.println("\nCara Stream API (Cari 'Erick'):");
        Mahasiswa targetStream = list.stream()
                .filter(m -> m.getNama().equalsIgnoreCase("Erick"))
                .findFirst()
                .orElse(null);
```

```
            System.out.println(targetStream != null ? targetStream : "Data tidak
        ditemukan");
        }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
=== 1. FILTERING (IPK >= 3.50) ===
Cara Konvensional:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

Cara Stream API:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

=== 2. SEARCHING / PENCARIAN TUNGGAL ===
Cara Konvensional (Cari 'Erick'):
Mhs[NIM=002, Nama=Erick , IPK=2,80]

Cara Stream API (Cari 'Erick'):
Mhs[NIM=002, Nama=Erick , IPK=2,80]
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  4.036 s
Finished at: 2026-11-10T14:11:42+07:00
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided 1

Seorang data scientist sedang membandingkan beberapa model *machine learning* untuk memprediksi data penjualan. Setiap model yang selesai dilatih akan dicatat hasilnya. Agar mudah dianalisis, ia meminta Anda membuat program Java sederhana untuk mengelola catatan hasil eksperimen tersebut.

Ketentuan :

1. Buatlah sebuah kelas untuk menyimpan hasil eksperimen satu model dengan atribut:
   - nama model (String)
   - akurasi (double, nilai 0 – 1)
   - waktu training dalam detik (int)

   Kelas ini harus memiliki *constructor*, *getter*, dan toString() dengan format: Model=<nama>, akurasi=<akurasi>, waktu=<waktu> detik
2. Kelas tersebut juga harus memiliki static variable yang menghitung jumlah eksperimen yang sudah dibuat (bertambah setiap kali objek baru dibuat), dan static method untuk mengambil nilai jumlah tersebut.
3. Kelas tersebut harus dapat diurutkan secara *default* berdasarkan akurasi dari yang tertinggi ke terendah, gunakan interface Comparable.
4. Buatlah sebuah kelas terpisah yang mengimplementasikan Comparator untuk mengurutkan hasil eksperimen berdasarkan waktu training dari yang tercepat ke terlama.
5. Pada *method main* (di kelas terpisah), simpan data berikut ke dalam sebuah List menggunakan generics:

Nama Model	Akurasi	Waktu Training (detik)
Logistic Regression	0.82	12
Random Forest	0.91	45
SVM	0.87	60
KNN	0.79	5
Decision Tree	0.84	8

6. Tampilkan hasil berikut secara berurutan:
   1. Jumlah eksperimen yang tercatat, gunakan *static method*.
   2. Daftar model yang diurutkan berdasarkan akurasi tertinggi** (Comparable).

3. Daftar model yang diurutkan berdasarkan **waktu training tercepat (Comparator).
4. Daftar model dengan akurasi minimal 0.85 (filtering menggunakan ekspresi lambda).
5. Data model dengan nama "KNN" (pencarian menggunakan ekspresi lambda, tampilkan null jika tidak ditemukan).

[EksperimenModel.java]

```
package Unguided;

import java.util.Comparator;

public class EksperimenModel implements Comparable<EksperimenModel> {
   private String namaModel;
   private double akurasi;
   private int waktuTraining; // dalam detik (ditambahkan tipe data int)

   // Static variable untuk menghitung jumlah eksperimen yang dibuat
   private static int jumlahEksperimen = 0;

   public EksperimenModel(String namaModel, double akurasi, int
waktuTraining) {
      this.namaModel = namaModel;
      this.akurasi = akurasi;
      this.waktuTraining = waktuTraining;
      jumlahEksperimen++; // bertambah setiap objek baru dibuat
   }

   public String getNamaModel() {
      return namaModel;
   }

   public double getAkurasi() {
      return akurasi;
   }

   public int getWaktuTraining() {
      return waktuTraining;
   }

   // Static method untuk mengambil nilai jumlah eksperimen
   public static int getJumlahEksperimen() {
      return jumlahEksperimen;
   }

   @Override
   public String toString() {
      return "Model=" + namaModel + ", akurasi=" + akurasi + ", waktu=" +
waktuTraining + " detik";
   }

   // Implementasi Comparable: Mengurutkan berdasarkan akurasi dari tertinggi
ke terendah (Descending)
   @Override
   public int compareTo(EksperimenModel o) {
      return Double.compare(o.akurasi, this.akurasi);
   }
```

```
}
```

[WaktuTrainingComparator.java]

```
package Unguided;
import java.util.Comparator;

public class WaktuTrainingComparator implements
Comparator<EksperimenModel> {
   @Override
   public int compare(EksperimenModel e1, EksperimenModel e2) {
      return Integer.compare(e1.getWaktuTraining(),
e2.getWaktuTraining());
   }
}
```

[Driver.java]

```
package Unguided;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

public class Driver {
   public static void main(String[] args) {
      // Menyimpan data ke dalam List menggunakan generics
      List<EksperimenModel> listEksperimen = new ArrayList<>();
      listEksperimen.add(new EksperimenModel("Logistic Regression",
0.82, 12));
      listEksperimen.add(new EksperimenModel("Random Forest", 0.91,
45));
      listEksperimen.add(new EksperimenModel("SVM", 0.87, 60));
      listEksperimen.add(new EksperimenModel("KNN", 0.79, 5));
      listEksperimen.add(new EksperimenModel("Decision Tree", 0.84, 8));

      // 1. Menampilkan jumlah eksperimen menggunakan static method
      System.out.println("Jumlah eksperimen: " +
EksperimenModel.getJumlahEksperimen());

      // 2. Menampilkan daftar model yang diurutkan berdasarkan akurasi
tertinggi (Comparable)
      Collections.sort(listEksperimen);
      System.out.println("Sorted by akurasi");
      listEksperimen.forEach(System.out::println);
```

```
        // 3. Menampilkan daftar model yang diurutkan berdasarkan waktu
    training tercepat (Comparator)
        listEksperimen.sort(new WaktuTrainingComparator());
        System.out.println("Sorted by waktu training");
        listEksperimen.forEach(System.out::println);

        // 4. Menampilkan daftar model dengan akurasi minimal 0.85
    (filtering menggunakan lambda)
        System.out.println("Model dengan akurasi minimal 0.85");
        List<EksperimenModel> filteredAkurasi = listEksperimen.stream()
            .filter(e -> e.getAkurasi() >= 0.85)
            .collect(Collectors.toList());
        filteredAkurasi.forEach(System.out::println);

        // 5. Pencarian model dengan nama "KNN" menggunakan lambda
    (tampilkan null jika tidak ditemukan)
        System.out.println("Pencarian model KNN");
        EksperimenModel hasilCari = listEksperimen.stream()
            .filter(e -> e.getNamaModel().equalsIgnoreCase("KNN"))
            .findFirst()
            .orElse(null);
        System.out.println(hasilCari);
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul9 ---
Jumlah eksperimen: 5
Sorted by akurasi
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=KNN, akurasi=0.79, waktu=5 detik
Sorted by waktu training
Model=KNN, akurasi=0.79, waktu=5 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model dengan akurasi minimal 0.85
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Pencarian model KNN
Model=KNN, akurasi=0.79, waktu=5 detik
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  5.817 s
Finished at: 2026-11-10T14:24:14+07:00
------------------------------------------------------------------------
```

Penjelasan :

Program Java yang dibuat pada praktikum ini dirancang untuk mengelola catatan hasil eksperimen dari beberapa model machine learning dengan memanfaatkan konsep static, collection, sorting, dan filtering. Program ini dibagi menjadi tiga kelas utama:

1. EksperimenModel.java: Berfungsi sebagai cetak biru (blueprint) objek eksperimen. Kelas ini menyimpan atribut seperti namaModel, akurasi, dan waktuTraining. Terdapat static variable (jumlahEksperimen) yang dikonfigurasi pada constructor untuk menghitung secara otomatis berapa banyak objek eksperimen yang telah diinisialisasi, serta static method (getJumlahEksperimen()) untuk mengambil nilai tersebut. Kelas ini juga mengimplementasikan interface Comparable dengan meng-override method compareTo agar objek dapat diurutkan secara default berdasarkan tingkat akurasi dari yang tertinggi ke terendah (descending).
2. WaktuTrainingComparator.java: Kelas terpisah yang mengimplementasikan interface Comparator untuk mendukung pengurutan alternatif berdasarkan durasi waktu training dari yang tercepat ke terlama (ascending) melalui method compare.

3. Driver.java: Kelas utama (main) yang menampung kumpulan data eksperimen ke dalam struktur data Collection berupa List dengan penerapan generics <EksperimenModel>. Kelas ini mengeksekusi berbagai fitur utama seperti menampilkan jumlah total eksperimen via static method, melakukan sorting data menggunakan Comparable dan Comparator, serta menyaring (filtering) data menggunakan Java Stream API dan ekspresi lambda (misalnya menyaring model dengan akurasi minimal 0.85 serta mencari model spesifik bernama "KNN").

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Konsep static (baik static variable maupun static method) memungkinkan variabel dan method untuk diakses secara langsung melalui kelasnya tanpa harus melakukan instansiasi objek baru, yang sangat berguna untuk melacak data global seperti penghitungan jumlah objek yang tercipta.
2. Collection Framework bersama generics menyediakan struktur data yang dinamis dan aman secara tipe data (type-safe) untuk menampung dan mengelola sekumpulan objek dalam jumlah banyak.
3. Implementasi interface Comparable memudahkan pengurutan alami (natural sorting) berdasarkan satu kriteria utama, sedangkan interface Comparator memberikan fleksibilitas untuk melakukan pengurutan kustom berdasarkan kriteria atau properti lainnya.
4. Penggunaan ekspresi lambda dan Stream API di dalam Java membuat proses penyaringan (filtering) dan pencarian data di dalam collection menjadi jauh lebih ringkas, elegan, dan efisien dibandingkan cara konvensional.